

C Program: 2D Array Menu-Driven Operations

Basic implementation covering Algorithm, Source Code, Test Cases, and Output Explanation

Problem Statement: Write a menu-driven C program to perform the following operations on a 2D array:

1. Find the sum of all elements in a matrix.
2. Find the sum of each row of a matrix and store it in an array.
3. Find the transpose of a matrix.
4. Exit.

1. Algorithm

1. **Start** the program.
2. Input the number of rows (r) and columns (c) for the matrix.
3. Input the elements of the 2D array (matrix) of size $r \times c$ using nested loops.
4. Display a menu with 4 options using a do-while loop and a switch-case structure.
5. **Option 1 (Sum of all elements):**
 - Initialize `total_sum = 0`.
 - Traverse each element of the matrix and add it to `total_sum`.
 - Print `total_sum`.
6. **Option 2 (Sum of each row):**
 - Declare a 1D array `row_sum[r]`.
 - For each row i , set `row_sum[i] = 0`.
 - Add all element values in row i to `row_sum[i]`.
 - Print the sum of each row individually and display the final `row_sum` array.
7. **Option 3 (Transpose of matrix):**
 - Iterate through columns first (j from 0 to $c-1$) and then rows (i from 0 to $r-1$).
 - Print element at index `[i][j]` to display the transposed matrix of size $c \times r$.
8. **Option 4 (Exit):** Terminate the program loop.
9. **End** the program.

2. Source Code (C Language)

```
#include <stdio.h>

int main() {
    int r, c;
    int i, j;
```

```

int choice;

// Input dimensions
printf("Enter the number of rows and columns: ");
scanf("%d %d", &r, &c);

int matrix[100][100];

// Input matrix elements
printf("Enter the matrix elements:
");
for (i = 0; i < r; i++) {
    for (j = 0; j < c; j++) {
        scanf("%d", &matrix[i][j]);
    }
}

// Menu-driven loop
do {
    printf("
--- MENU ---
");
    printf("1. Find the sum of all elements in a matrix
");
    printf("2. Find the sum of each row of a matrix and store it in an array
");
    printf("3. Find the transpose of a matrix
");
    printf("4. Exit
");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: {
            int total_sum = 0;
            for (i = 0; i < r; i++) {
                for (j = 0; j < c; j++) {
                    total_sum += matrix[i][j];
                }
            }
            printf("Sum of all elements = %d
", total_sum);
            break;
        }
        case 2: {
            int row_sum[100];
            printf("Row sums:
");
            for (i = 0; i < r; i++) {
                row_sum[i] = 0;
                for (j = 0; j < c; j++) {
                    row_sum[i] += matrix[i][j];
                }
                printf("Sum of row %d = %d
");
            }
        }
    }
} while (choice != 4);

```

```

", i + 1, row_sum[i]);
    }

    // Displaying row-sum array
    printf("The row-sum array contains: [");
    for (i = 0; i < r; i++) {
        printf("%d", row_sum[i]);
        if (i < r - 1) {
            printf(", ");
        }
    }
    printf("]
");
    break;
}
case 3: {
    printf("Transpose of the matrix:
");
    for (j = 0; j < c; j++) {
        for (i = 0; i < r; i++) {
            printf("%d ", matrix[i][j]);
        }
        printf("
");
    }
    break;
}
case 4: {
    printf("Exiting program...
");
    break;
}
default: {
    printf("Invalid choice! Please try again.
");
}
}
} while (choice != 4);

return 0;
}

```

3. Test Case / Sample Execution

Input Data:

- Rows and Columns: 2 3
- Matrix Elements:

```

1 2 3
4 5 6

```

```
Enter the number of rows and columns: 2 3
Enter the matrix elements:
1 2 3
4 5 6
```

```
Enter your choice: 1
Sum of all elements = 21
```

```
Enter your choice: 2
Row sums:
Sum of row 1 = 6
Sum of row 2 = 15
The row-sum array contains: [6, 15]
```

```
Enter your choice: 3
Transpose of the matrix:
1 4
2 5
3 6
```

```
Enter your choice: 4
Exiting program...
```

4. Explanation of Key Concepts (Beginner Level)

- **2D Array (Matrix):** Represented using `matrix[100][100]` where elements are accessed via row index `i` and column index `j`.
- **Menu-driven structure:** Built using a do-while loop along with switch-case to keep asking for user choices until option 4 (Exit) is selected.
- **Row Sum Array:** A separate 1D array `row_sum[r]` stores the sum of each row, which is calculated by looping through each column in that row.
- **Transpose:** Transposing changes rows into columns. By swapping the loop order (outer loop for columns `j`, inner loop for rows `i`), we display the transpose directly without extra memory allocation.